

Herald

Каждое оповещение достигает нужного адресата — без необходимости запоминать синтаксис команд.

Краткое описание

Herald принимает системные события и надёжно распределяет их по нескольким каналам оповещений, чтобы каждое сообщение доходило до нужного получателя. Подписчики взаимодействуют на естественном языке, а Herald определяет их намерения по трёхступенчатой схеме: быстрый путь команд → вывод намерений через LLM → уточняющий запрос в случае неясности.

Описание

Herald — это надёжная инфраструктура оповещений, гарантирующая, что системное событие дойдёт до человека, который сможет на него отреагировать. Это неприметный, но критически важный слой, где большинство самодельных систем оповещений терпят неудачу. Он принимает события и надёжно распределяет их по нескольким каналам, исключая типичные сбои: потерю сообщения, отправку в неактивный канал или его погребение под потоком шума до того, как оно утратит актуальность. Однако надёжная доставка — это лишь половина дела. Другая половина — то, как человек реагирует на оповещение. Здесь Herald отказывается от привычного подхода, когда пользователям приходится заучивать жёсткий синтаксис команд для взаимодействия с ботом. Подписчики просто пишут на естественном языке, а Herald определяет их намерения по продуманной трёхступенчатой схеме: быстрый путь распознаёт явные команды мгновенно, затем вступает механизм вывода намерений на основе LLM (через Claude Code) для свободных формулировок, а в случае настоящей неопределённости срабатывает уточняющий запрос, который отвечает, помечает сообщение и задаёт вопрос. Эта лестница «распознать → вывести → уточнить» — квинтэссенция всей философии дизайна: стандартные случаи обрабатываются мгновенно и детерминированно, гибкие — с помощью модели,

а неопределённые никогда не решаются наугад, рискуя запустить неверное действие. Кроме того, Herald отслеживает участие и атрибуцию: переменная окружения с именем оператора (HERALD_<CHANNEL>_OPERATOR_USERNAME) и контракт участников/атрибуции управляют полями created_by/assigned_to и упоминаниями в оповещениях, так что всегда ясно, кто что сделал и кого уведомляют. С точки зрения управления, Herald наследует Helix Constitution как встроенный подмодуль и следует его правилам, а также является одним из первых производственных потребителей Docs Chain — весь его корпус из 66 документов в формате Markdown→HTML/PDF/DOCX проходит через exes:-трансформации Docs Chain и проверяется на корректность. Herald в основном представляет собой инструментарий Shell/Go с многоуровневыми спецификациями (последовательное замещение версий V1→V2→V3→V4) и руководствами по настройке операторов для мессенджеров и диспетчеров LLM/агентов.

Зачем мы его создали

Оповещения часто сбоят незаметно: отправляются не туда, теряются или требуют жёсткого синтаксиса команд, который пользователи не запомнят. Herald создан для того, чтобы гарантировать надёжное распределение сообщений и позволить людям реагировать на естественном языке — так оповещения станут не только надёжными, но и простыми в использовании.

Почему это меняет правила игры

Система объединяет два компонента, которые обычно приобретаются по отдельности, — надёжную многоканальную маршрутизацию событий и интерфейс на естественном языке — в единое решение, где операторам достаточно просто говорить, а программа сама понимает, что они имеют в виду. Но именно механизм уточнения делает её надёжной в промышленной эксплуатации: система оповещения, которая предпочитает переспросить, а не выдать ложное срабатывание, — это та система, которую можно без опасений подключать к реальным процессам.

Что нового

- Трёхступенчатая дисциплина обработки намерений: быстрый путь команды → вывод LLM → уточнение с запросом.
- Взаимодействие с подписчиками на естественном языке (не нужно учить синтаксис команд).

- Контракт атрибуции участников, обеспечивающий поля `created_by/assigned_to` и упоминания через `@`.
- Настоящий потребитель Docs Chain (корпус из 66 документов, мультимедийный, верифицированный).

Проблемы и решения

- **Неоднозначность намерений в естественном языке:** решено с помощью трёхступенчатой лестницы распознавания/вывода/уточнения вместо слепого угадывания.
- **Надёжное разветвление:** решено за счёт архитектуры «приём → многоканальная диспетчеризация», чтобы оповещения доходили до нужного адресата.
- **Корректная атрибуция по каналам:** решено с помощью переменной окружения с именем оператора и контракта атрибуции участников.
- **Расхождение документации:** решено путём интеграции корпуса документации в Docs Chain с верифицированными преобразованиями.

Технологический стек (зачем и как)

- **Go** — ядро логики событий и диспетчеризации (с учётом языковых паттернов организации).
- **Shell** — инструменты для операторов и скрипты настройки.
- **Код Claude (LLM)** — уровень вывода намерений для сообщений в свободной форме.
- **Адаптеры каналов Messenger** — многоканальное разветвление уведомлений.
- **Docs Chain** — конвейер сборки и верификации документации (Markdown → HTML/PDF/DOCX).
- **Подмодуль Helix Constitution** — унаследованное управление и правила.